
CSE 151B/251B Milestone Report: Inference-Time Engineering for Math Reasoning with a Quantized Thinking LLM

Connor Wu

Department of Computer Science and Engineering
University of California, San Diego
cow003@ucsd.edu

Abstract

The CSE 151B Spring 2026 competition asks us to maximise accuracy on 1126 mixed multiple-choice (MCQ) and free-form math problems using a fixed, INT8-quantised Qwen3-4B-Thinking-2507 model. With no training in our current setup, all gains come from inference-time engineering. We extend the starter notebook with four targeted changes – LaTeX repair scoped to math delimiters, prompts tuned for a thinking model, a lazy-fallback MCQ extractor with logprobs calibration, and hardened brace-aware boxed-answer parsing – each with negligible runtime cost. **Final private Kaggle score: TODO – to be added after the full submission is graded.**

1 Introduction

Problem Definition. The public split contains $n=1126$ items: 375 MCQs with up to ten labelled options $A, B, \dots, J$, and 751 free-form problems whose gold answers are symbolic or numeric expressions. The model Qwen3-4B-Thinking-2507 is served by vLLM under INT8 BitsAndBytes quantisation; no fine-tuning is part of the current pipeline. A symbolic judger checks free-form answers for mathematical equivalence rather than string match.

Problem Significance. Quantised inference of mid-sized open-weight reasoning models is the deployment setting most practitioners actually face: training budgets are out of reach, but a single consumer GPU can hold a 4–7B model in 8-bit. Understanding which inference-time choices help – particularly for “thinking” models that emit a long `<think> ... </think>` chain before the answer – transfers directly to real low-compute applications.

Technical Challenge. Three challenges shape the project. (1) *Data quality*: the dataset stores LaTeX with backslashes stripped (`frac`, `infty`, `int`, etc.), and a naive repair corrupts common English words (“sum”, “times”, “log”) that share spellings with LaTeX commands. (2) *Thinking-model output handling*: the model emits a long internal chain before the final boxed answer; the token budget and stop conditions must accommodate it. (3) *Robust scoring*: MCQ scoring is letter match, but free-form scoring requires the response to contain exactly one well-formed `\boxed{}` that survives nested-brace parsing.

Contributions. The starter code is a working end-to-end pipeline (vLLM, 32k-token budget, simple system prompts, regex letter extraction, symbolic judger). Its limitations: no LaTeX repair, MCQ extraction never abstains (it falls back to the last capital letter anywhere in the response), and `\boxed{}` contents with nested braces are truncated. Our changes:

- **Scoped LaTeX repair** that restores backslashes only inside $\$. . . \$$ math blocks, leaving English untouched.
- **Thinking-tuned prompts** – the MCQ prompt explicitly directs the model to emit `\boxed{X}` after `</think>`, and the free-form prompt includes two in-context examples enforcing a single boxed answer with comma-separated sub-answers.
- **Lazy-fallback MCQ extractor** – the thinking response is the primary source; a cheap no-think logprobs pass over letter tokens is invoked only when the primary extractor abstains.
- **Hardened answer parsing** – `extract_letter` reads preferentially from the post-`</think>` region, and `extract_boxed` uses a brace-counting parser so that `\boxed{\frac{1}{2}}` extracts in full.

2 Related Work

Reasoning LLMs. Recent open-weight “thinking” models such as the Qwen3 series [3] interleave a hidden chain of thought, delimited by `<think></think>`, with a final user-facing answer. Their accuracy on math benchmarks improves with adequate decoding budget and suitable sampling temperature; recommended defaults are documented by the model authors.

Inference serving. vLLM [2] provides batched, paged attention serving of HuggingFace models, including BitsAndBytes 8-bit weight quantisation. Decoding throughput depends on `max_num_seqs`, `enable_prefix_caching`, and whether CUDA graphs are enabled (`enforce_eager=False`).

Symbolic answer judging. Math benchmarks such as MATH [1] have driven the design of symbolic equivalence checkers built on SymPy-like CAS backends; the course-provided `Judger` class follows this lineage and accepts free-form responses, extracting and normalising the boxed expression internally.

3 Methods

Problem Setting. Let $\mathcal{D} = \{(q_i, O_i, y_i)\}_{i=1}^n$ be the public set with O_i possibly empty. The model f_θ is held fixed; we choose a prompt template π , sampling parameters σ , and extraction function g to maximise

$$\text{Acc}(\pi, \sigma, g) = \frac{1}{n} \sum_{i=1}^n \mathbf{1}[\text{Judge}(g(f_\theta(\pi(q_i, O_i); \sigma))) = y_i],$$

with `Judge` being letter equality for MCQ and symbolic equivalence for free-form. There is no trainable loss in this milestone; all optimisation is over the discrete choices (π, σ, g) .

Changes from the starter. Table 1 summarises the differences between the starter and our current optimised notebook. Each change is small and local, and the lazy-fallback design means the optimised version costs no more at decode time than the starter.

4 Experiments

4.1 Baselines

We compare three configurations on the public development data:

- **Starter** – the unmodified course-provided notebook.
- **Optimised** – this milestone, with the changes summarised in Table 1.

4.2 Evaluation

MCQ predictions are scored by case-insensitive letter equality with the gold label. Free-form predictions are scored by the course-provided symbolic `Judger`, which extracts and normalises the boxed expression and tests mathematical equivalence against each gold answer. We report MCQ, free-form, and overall accuracy.

Table 1: Differences between the starter and the current optimised notebook. All changes are inference-time only; the model is unchanged.

Component	Starter	Optimised
Input preprocessing	None.	LaTeX repair scoped to $\$...\$$ blocks: restores backslashes on <code>frac</code> , <code>infty</code> , <code>int</code> , <code>sum</code> , ... inside math while leaving English untouched.
MCQ prompt	Generic “output the letter in <code>\boxed{}</code> .”	Tuned for a thinking model: “After <code></think></code> , output exactly one line: <code>\boxed{X}</code> .”
Free-form prompt	Generic step-by-step instruction.	Adds two in-context examples enforcing a single <code>\boxed{}</code> with comma-separated sub-answers.
MCQ extraction	Regex that falls back to the last capital letter anywhere in the response (never abstains).	Multi-pattern extractor that prefers text after <code></think></code> and abstains otherwise; on abstention, a cheap no-think logprobs pass reads $P(\text{letter})$ over $A-J$ tokens.
Free-form parsing	Raw response sent to the judge.	Same – plus a brace-counting <code>extract_boxed</code> used for diagnostics.
Sampling / budget	$T=0.6$, $p=0.95$, $k=20$, <code>max_tokens</code> = 32768, <code>enable_prefix_caching=False</code> .	Same sampling; <code>max_tokens</code> =16384 (ample in practice, lighter on the KV cache); <code>enable_prefix_caching=True</code> .

4.3 Implementation details

The model is Qwen/Qwen3-4B-Thinking-2507 served by vLLM 0.20.2 with `quantization="bitsandbytes"`, `max_model_len=16384`, `gpu_memory_utilization=0.50`. Sampling follows the Qwen3-Thinking recommendation ($T=0.6$, $p=0.95$, $\text{top}_k=20$, `repetition_penalty=1.0`), with `max_tokens=16,384` on the main pass and a near-greedy fallback ($T=0.01$, $p=1.0$, `logprobs=20`) for MCQ logprobs. GPU footprint at load is ≈ 2.7 GiB of weights plus an 8.3 GiB KV cache, fitting in ~ 16 GiB of VRAM.

Add public GitHub URL for final report.

4.4 Results

Table 2 reports the starter’s preliminary $N=5$ smoke-test accuracy (its default), recorded directly from the notebook output. A full $N=1126$ run of the optimised notebook is in progress; final numbers will replace the placeholder row in the camera-ready version of this report.

Table 2: Preliminary accuracy on the public set. The starter cell runs only the first 5 items by default. Optimised numbers are pending the full-dataset run.

Setting	MCQ	Free-form	Overall
Starter ($N=5$)	1/2 (50.0%)	2/3 (66.7%)	3/5 (60.0%)
Optimised ($N=1126$)	TODO – full-dataset run pending		

The optimised configuration is no more expensive than the starter at decode time: `max_tokens` is lower, the fallback calibration pass is invoked only for MCQs where the primary extractor abstains, and `enable_prefix_caching=True` allows the system prompt to be reused across the batch.

5 Discussion

Thinking models are sensitive to anything that prevents the final answer from surfacing: token budgets that truncate the chain, scoring paths that prefer a no-think pass, and `stop=[...]` lists that can fire

during reasoning all act as silent answer-suppressors. The optimised notebook addresses these failure modes upfront.

Achievements. A small, runtime-flat delta to the starter that adds scoped LaTeX repair, thinking-tuned prompts, a lazy-fallback MCQ extractor with logprobs calibration, and brace-aware boxed parsing.

Bottlenecks and Mitigation. The dominant bottleneck is decode latency: a full 1126-question run takes a non-trivial fraction of an hour, which makes hyperparameter sweeps expensive. We plan to maintain a frozen ~ 100 -item stratified subsample (75 free-form, 25 MCQ) as a stable evaluation surface for fast iteration.

Plans Forward. Three directions are queued: (i) self-consistency over $n=5$ free-form samples scored by the symbolic judger; (ii) a cheap no-think first pass used to *filter* which items deserve the expensive thinking pass on rerun; (iii) a post-processor that normalises boxed expressions to make the judger robust on borderline cases.

Team Responsibilities. Solo submission. Connor Wu is responsible for the full pipeline, evaluation harness, and report.

Timelines.

- Weeks 1–3: environment setup, starter notebook end-to-end, dataset and judger inspection. (*Complete.*)
- Weeks 4–5: optimised notebook (this milestone). (*Complete.*)
- Week 6: full $N=1126$ evaluation and first leaderboard submission.
- Week 7: self-consistency / majority voting on free-form.
- Week 8: cheap-first-pass filtering of items for the thinking pass.
- Week 9: post-processing pipeline (numeric normalisation).
- Week 10: final ablations and submission.

Adjust once final-quarter dates are confirmed.

LLM Usage

Anthropic’s Claude (model `claude-opus-4-7`, via the Claude desktop “Cowork” interface) was used as an interactive coding assistant with read/write access to this repository. Claude reviewed the diff between the starter and the optimised notebook, proposed and applied the changes summarised in Table 1, and drafted this report from a short briefing on scope, authorship, and which numbers to cite. The author ran all experiments, verified the patched notebook, and reviewed and revised every claim in the report. The author takes full responsibility for the contents.

References

- [1] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2021.
- [2] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP)*, 2023.
- [3] An Yang, Anfeng Li, Baosong Yang, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.